Лекция: От SOLID к шаблонам проектирования и архитектуре систем

Цель занятия: Изучить способы решения типовых задач проектирования, научиться бороться с раздутыми условными операторами и освоить концепцию «Чистой архитектуры» на уровне моделирования данных.

---

1. Наследование vs Композиция: Почему иерархии — это ловушка

На первых курсах студентов учат, что наследование — это база. На 3 курсе пора узнать, что **композиция почти всегда лучше**.

Теория:

Глубокие иерархии наследования создают «хрупкие базовые классы». Изменение в корневом классе может сломать десятки потомков. Композиция позволяет собирать поведение объекта из независимых частей во время выполнения (Runtime).

Пример (C#): Проблема «Боевого робота»

Представьте игру. У нас есть `Robot`. Мы хотим робота, который стреляет, и робота, который летает.

Плохой путь (Наследование):
`FlyingRobot`, `ShootingRobot`... А если нужен `FlyingShootingRobot`? Возникает проблема ромбовидного наследования или дублирования кода.

Правильный путь (Композиция/Стратегия):

```
public interface IMovement { void Move(); }
public interface IWeapon { void Fire(); }

public class Walking : IMovement { public void Move() =>
Console.WriteLine("Иду на двоих..."); }
public class Flying : IMovement { public void Move() =>
Console.WriteLine("Лечу над землей..."); }
```

```
public class Laser : IWeapon { public void Fire() => Console.WriteLine("Пиу-пиу лазером!"); }

public class Robot {
    private IMovement _movement;
    private IWeapon _weapon;

    public Robot(IMovement movement, IWeapon weapon) {
        _movement = movement;
        _weapon = weapon;
    }

    public void Action() {
        _movement.Move();
        _weapon.Fire();
    }
}

// Использование: мы можем "собрать" любого робота без создания новых классов
var terminatror = new Robot(new Walking(), new Laser());
```

---

2. Поведенческие паттерны: Избавляемся от `if-else` и `switch`

Если в вашем коде есть `switch` на 10 кейсов — это сигнал, что пора внедрять паттерны.

2.1. Паттерн "State" (Состояние)

Проблема: Поведение объекта зависит от его внутреннего состояния, и код превращается в лабиринт условий.
Пример: Заказ в интернет-магазине (`New`, `Paid`, `Shipped`, `Cancelled`).

```
    public abstract class OrderState {
        public abstract void Next(Order order);
        public abstract void Cancel(Order order);
    }

    public class NewState : OrderState {
        public override void Next(Order order) => order.SetState(new PaidState());
```

```
        public override void Cancel(Order order) => order.SetState(new
CancelledState());
    }

    public class Order {
        private OrderState _state = new NewState();
        public void SetState(OrderState state) => _state = state;
        public void Next() => _state.Next(this);
        public void Cancel() => _state.Cancel(this);
    }
```

2.2. Паттерн "Strategy" (Стратегия)

*Уже был затронут в примере с роботами, но важно подчеркнуть: Стратегия меняет **алгоритм**, Состояние меняет **всю логику объекта**.*

---

3. Структурные паттерны: Декоратор и Прокси

3.1. Decorator (Обертка)

Позволяет динамически добавлять объектам новые обязанности, не меняя их код.

Задача для рефакторинга: Система уведомлений. Сначала слали только Email, потом добавили SMS, потом Telegram.
Решение:

```
    public interface INotifier { void Send(string message); }

    public class EmailNotifier : INotifier {
        public void Send(string message) => Console.WriteLine($"Email:
{message}");
    }

    // Базовый декоратор
    public abstract class NotifierDecorator : INotifier {
        protected INotifier _wrapped;
        public NotifierDecorator(INotifier n) => _wrapped = n;
        public virtual void Send(string message) =>
_wrapped.Send(message);
    }
```

```
public class SmsDecorator : NotifierDecorator {
    public SmsDecorator(INotifier n) : base(n) {}
    public override void Send(string message) {
        base.Send(message); // Сначала шлем Email
        Console.WriteLine($"SMS: {message}"); // Потом SMS
    }
}

// Использование
INotifier myChain = new SmsDecorator(new EmailNotifier());
myChain.Send("Ваш заказ готов!");
```

---

4. Введение в моделирование: Value Objects vs Entities (DDD)

Это критически важная тема для 3 курса. Студенты часто используют примитивы (`int`, `string`) там, где нужны объекты.

Теория:

1. **Entity (Сущность):** Имеет уникальный ID (например, Пользователь). Даже если два пользователя имеют одинаковые имена, они разные.
2. **Value Object (Объект-значение):** Не имеет ID. Определяется своими свойствами. Если у двух объектов `Money` одинаковая сумма и валюта — это один и тот же объект. **Value Objects должны быть Immutable (неизменяемыми).**

Плохой код:

```
public void Transfer(decimal amount, string currency) { ... }
// Ошибка: можно случайно передать USD в метод, ждущий EUR.
```

Хороший код (Value Object):

```
public record Money(decimal Amount, string Currency) {
    public Money Add(Money other) {
```

```
            if (Currency != other.Currency) throw new Exception("Разные
валюты!");
            return new Money(Amount + other.Amount, Currency);
        }
    }
```

---

5. Обработка ошибок: Result Pattern вместо Exceptions

Использование `try-catch` для бизнес-логики — это дорого и делает код нечитаемым.

Концепция: Метод возвращает объект, который говорит: «Я либо успешно вернул данные, либо вот ошибка».

```
    public class Result<T> {
        public bool IsSuccess { get; }
        public T Value { get; }
        public string Error { get; }

        private Result(T value, bool success, string error) {
            Value = value; IsSuccess = success; Error = error;
        }

        public static Result<T> Success(T value) => new Result<T>(value, true,
null);
        public static Result<T> Failure(string error) => new
Result<T>(default, false, error);
    }

    // Применение
    public Result<User> Register(string email) {
        if (string.IsNullOrWhiteSpace(email)) return
Result<User>.Failure("Email пуст");
        return Result<User>.Success(new User(email));
    }
```

Практическая часть (Воркшоп на 1.5 - 2 часа)

Задача 1: "Система лояльности" (Рефакторинг Strategy)

Дано: Метод `CalculateDiscount`, в котором 15 `if-else` для разных типов клиентов (VIP, Пенсионер, Студент, Новый, "Черная пятница").
Задание: Реализовать через паттерн Стратегия так, чтобы новые скидки можно было добавлять, создавая новый класс, а не меняя старый код.

Задача 2: "Банковская транзакция" (Value Objects)

Дано: Класс `Account` с полями `decimal Balance`, `string Currency`.
Задание:

1. Создать Value Object `Money`.
2. Реализовать метод `Deposit(Money amount)`, который проверяет соответствие валют.
3. Сделать так, чтобы `Money` нельзя было изменить после создания.

Задача 3: "Умный дом" (State)

Дано: Класс `Heater` (Обогреватель). Состояния: `Off`, `Waiting` (ждем датчик), `Heating` (греем), `Overheated` (блокировка).
Задание: Реализовать переходы между состояниями без единого `if`.

Дополнительная теория: Анемичная vs Богатая модель данных

Многие пишут классы, где только свойства (Get/Set) — это **Anemic Domain Model** (антипаттерн в сложном ООП).
В **Rich Domain Model** (богатой модели) логика валидации и изменения состояния лежит внутри класса.

Пример для обсуждения:
Должен ли класс `User` иметь метод `ChangePassword(string newPass)` или это должен делать `UserService`?
Ответ в рамках современного ООП: Если пароль требует внутренней валидации (не пустой, не старый), то лучше инкапсулировать это в классе `User`.